

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский университет ИТМО**

**Отчет по лабораторной работе №4
«ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ.
ПРЕДСТАВЛЕНИЯ. РАБОТА С ИНДЕКСАМИ»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Буй Тхук Хуен
Факультет прикладной информатики - **Группа** К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна
Белов Александр Олегович

Санкт-Петербург
2026 г.

1. Цель работы

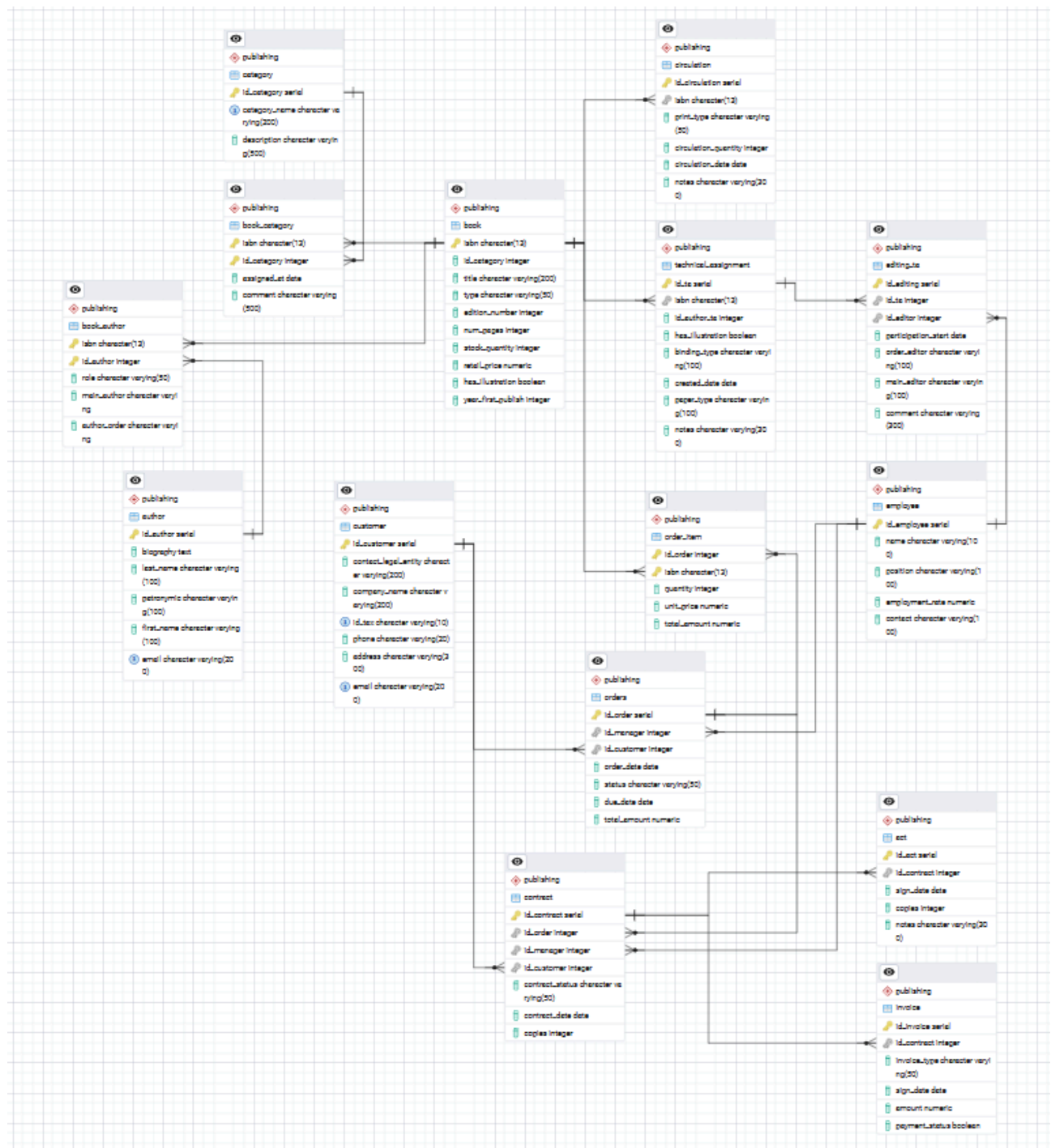
Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2. Практическое задание

- a. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
- b. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
- c. Изучить графическое представление запросов и просмотреть историю запросов.
- d. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

3. Схема базы данных

Вариант 5 - БД «Издательство компьютерной литературы»



4. Выполнение

4.1 Создать запросы и представления на выборку данных к базе данных PostgreSQL.

- На этапе были заполнены данными с использованием SQL-запросов.
- При заполнении учитывались:
 - соблюдение ограничений (constraints);
 - корректные связи между таблицами;
 - реалистичность данных.

+ Таблица category:

Processes x Untitled* x publishing_book/postgres@PostgreSQL 18* x

publishing_book/postgres@PostgreSQL 18

Query Query History

```
1 INSERT INTO publishing.category (  
2     id_category,  
3     category_name,  
4     description  
5 )  
6 VALUES  
7 (1, 'Programming', 'Books about programming'),  
8 (2, 'Databases', 'Database systems'),  
9 (3, 'AI', 'Artificial Intelligence'),  
10 (4, 'Networking', 'Networks'),  
11 (5, 'Security', 'Cybersecurity');
```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 61 msec.

Data Output Messages Notifications			
Showing rows: 1 to 5 of 1			
	id_category [PK] integer	category_name character varying (200)	description character varying (500)
1	1	Programming	Books about programm...
2	2	Databases	Database systems
3	3	AI	Artificial Intelligence
4	4	Networking	Networks
5	5	Security	Cybersecurity

+ Таблица author:

Query

Query History

```
1  INSERT INTO publishing.author (
2      id_author,
3      biography,
4      last_name,
5      patronymic,
6      first_name,
7      email
8  )
9  SELECT
10     i,
11     'Biography of author ' || i,
12     'LastName' || i,
13     'MiddleName' || i,
14     'FirstName' || i,
15     'author' || i || '@mail.com'
16 FROM generate_series(1, 200) AS s(i);
```

Data Output

Messages

Notifications

INSERT 0 200

Query returned successfully in 97 msec.

	id_author [PK] integer	biography text	last_name character varying (100)	patronymic character varying (100)	first_name character varying (100)	email character varying (200)
1	1	Biography of author 1	LastName1	MiddleName1	FirstName1	author1@mail.com
2	2	Biography of author 2	LastName2	MiddleName2	FirstName2	author2@mail.com
3	3	Biography of author 3	LastName3	MiddleName3	FirstName3	author3@mail.com
4	4	Biography of author 4	LastName4	MiddleName4	FirstName4	author4@mail.com
5	5	Biography of author 5	LastName5	MiddleName5	FirstName5	author5@mail.com
6	6	Biography of author 6	LastName6	MiddleName6	FirstName6	author6@mail.com
7	7	Biography of author 7	LastName7	MiddleName7	FirstName7	author7@mail.com
8	8	Biography of author 8	LastName8	MiddleName8	FirstName8	author8@mail.com
9	9	Biography of author 9	LastName9	MiddleName9	FirstName9	author9@mail.com
10	10	Biography of author ...	LastName10	MiddleName10	FirstName10	author10@mail.com

+ Таблица book:

Data Output Messages Notifications										
Showing rows: 1 to 10 Page No: 1										
	isbn [PK] character varying	id_category integer	title character varying (200)	type character varying (50)	edition_number integer	num_pages integer	stock_quantity integer	retail_price numeric	has_illustration boolean	year_first_publish integer
1	ISBN-1	5	Book Title 1	учебник	2	108	55	577.36	false	2017
2	ISBN-2	1	Book Title 2	учебное пособие	4	511	34	352.72	false	2021
3	ISBN-3	3	Book Title 3	учебное пособие	3	170	98	133.69	true	2011
4	ISBN-4	5	Book Title 4	учебное пособие	3	229	45	592.87	false	2015
5	ISBN-5	5	Book Title 5	учебник	4	569	92	335.41	false	2014
6	ISBN-6	4	Book Title 6	учебник	5	158	39	641.37	false	2016
7	ISBN-7	5	Book Title 7	учебник	2	521	23	742.75	true	2005
8	ISBN-8	2	Book Title 8	учебное пособие	2	169	47	857.46	true	2021
9	ISBN-9	4	Book Title 9	учебное пособие	2	547	88	936.99	true	2015
10	ISBN-10	3	Book Title 10	учебное пособие	4	556	103	134.21	true	2024

+ Таблица book_author:

Query Query History	
<pre> 1 INSERT INTO publishing.book_author (2 isbn, 3 id_author, 4 role, 5 main_author, 6 author_order 7) 8 SELECT 9 'ISBN-' (random()*499 + 1)::int, 10 (random()*199 + 1)::int, 11 (ARRAY['автор', 'редактор'])[floor(random()*2+1)], 12 'Yes', 13 (random()*3 + 1)::int 14 FROM generate_series(1, 2000) 15 ON CONFLICT (isbn, id_author) DO NOTHING; </pre>	
Data Output Messages Notifications	
INSERT 0 1981	
Query returned successfully in 119 msec.	

Query

Query History

1

SELECT * FROM publishing.book_author LIMIT 10;

Data Output

Messages

Notifications

<

+ Таблица book_category:

Query

Query History

1

SELECT * FROM publishing.book_category LIMIT 10;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	isbn [PK] character (13)	id_category [PK] integer	assigned_at date	comment character varying (500)
1	ISBN-1	1	2024-01-28	Assigned automatically
2	ISBN-1	2	2024-06-13	Assigned automatically
3	ISBN-1	4	2024-11-03	Assigned automatically
4	ISBN-2	1	2024-06-14	Assigned automatically
5	ISBN-2	3	2025-06-29	Assigned automatically
6	ISBN-4	1	2023-11-15	Assigned automatically
7	ISBN-4	2	2024-07-19	Assigned automatically
8	ISBN-5	2	2024-11-18	Assigned automatically
9	ISBN-5	3	2023-09-15	Assigned automatically
10	ISBN-6	2	2025-08-22	Assigned automatically

+ Таблица customer:

Query	Query History
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18	<pre>INSERT INTO publishing.customer (id_customer, contact_legal_entity, company_name, id_tax, phone, address, email) SELECT i, 'Contact Person ' i, 'Company ' i, LPAD(i::text, 10, '0'), '+84-900-' LPAD(i::text, 4, '0'), 'Address ' i, 'customer' i '@mail.com' FROM generate_series(1, 200) AS s(i);</pre>
Data Output	Messages
INSERT 0 200	
Query returned successfully in 77 msec.	

Query

Query History

1

SELECT * FROM publishing.customer LIMIT 10;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

	id_customer [PK] integer	contact_legalEntity character varying (200)	company_name character varying (200)	id_tax character varying (10)	phone character varying (20)	address character varying (300)	email character varying (200)
1	1	Contact Person 1	Company 1	0000000001	+84-900-0001	Address 1	customer1@mail.com
2	2	Contact Person 2	Company 2	0000000002	+84-900-0002	Address 2	customer2@mail.com
3	3	Contact Person 3	Company 3	0000000003	+84-900-0003	Address 3	customer3@mail.com
4	4	Contact Person 4	Company 4	0000000004	+84-900-0004	Address 4	customer4@mail.com
5	5	Contact Person 5	Company 5	0000000005	+84-900-0005	Address 5	customer5@mail.com
6	6	Contact Person 6	Company 6	0000000006	+84-900-0006	Address 6	customer6@mail.com
7	7	Contact Person 7	Company 7	0000000007	+84-900-0007	Address 7	customer7@mail.com
8	8	Contact Person 8	Company 8	0000000008	+84-900-0008	Address 8	customer8@mail.com
9	9	Contact Person 9	Company 9	0000000009	+84-900-0009	Address 9	customer9@mail.com
10	10	Contact Person 10	Company 10	0000000010	+84-900-0010	Address 10	customer10@mail.com

+ Таблица employee:

Query	Query History
1	INSERT INTO publishing.employee (
2	id_employee,
3	name,
4	position,
5	employment_rate,
6	contact
7	)
8	SELECT
9	i,
10	'Employee ' i,
11	(ARRAY['менеджер', 'редактор', 'бухгалтер'])[floor(random()*3+1)],
12	(random()*2 + 0.5)::numeric(2,1),
13	'contact' i
14	FROM generate_series(1, 50) AS s(i);

Data Output	Messages	Notifications
INSERT 0 50		
Query returned successfully in 74 msec.		

Query Query History

```
1 SELECT * FROM publishing.employee LIMIT 10;
```

Data Output Messages Notifications

SQL

	id_employee [PK] integer	name character varying (100)	position character varying (100)	employment_rate numeric	contact character varying (100)
1	1	Employee 1	бухгалтер	1.8	contact1
2	2	Employee 2	бухгалтер	0.6	contact2
3	3	Employee 3	редактор	1.6	contact3
4	4	Employee 4	менеджер	1.9	contact4
5	5	Employee 5	редактор	0.7	contact5
6	6	Employee 6	редактор	0.6	contact6
7	7	Employee 7	бухгалтер	0.5	contact7
8	8	Employee 8	менеджер	1.1	contact8
9	9	Employee 9	бухгалтер	1.3	contact9
10	10	Employee 10	менеджер	1.0	contact10

+ Таблица orders:

Query Query History

```
1 INSERT INTO publishing.orders (  
2     id_order,  
3     id_manager,  
4     id_customer,  
5     order_date,  
6     status,  
7     due_date,  
8     total_amount  
9 )  
10 SELECT  
11     i,  
12     (random()*49 + 1)::int, -- employee 1→50  
13     (random()*199 + 1)::int, -- customer 1→200  
14     CURRENT_DATE - (random()*30)::int,  
15     (ARRAY['получено', 'обработано', 'завершено', 'доставлено', 'отменено'])[floor(random()*5+1)],  
16     CURRENT_DATE + (random()*30)::int,  
17     (random()*10000 + 100)::numeric(10,2)  
18 FROM generate_series(1, 300) AS s(i);
```

Data Output Messages Notifications

INSERT 0 300

Query returned successfully in 72 msec.

Query Query History

```
1 SELECT * FROM publishing.orders LIMIT 10;
```

Data Output Messages Notifications

	id_order [PK] integer	id_manager integer	id_customer integer	order_date date	status character varying (50)	due_date date	total_amount numeric
1	1	46	189	2026-04-22	завершено	2026-05-19	9718.77
2	2	4	24	2026-04-20	отменено	2026-05-03	6334.91
3	3	33	153	2026-04-21	доставлено	2026-04-23	1455.78
4	4	36	55	2026-03-31	обработано	2026-05-06	1282.17
5	5	24	160	2026-04-05	отменено	2026-05-03	3292.20
6	6	12	15	2026-04-23	доставлено	2026-05-20	5826.91
7	7	13	196	2026-03-26	получено	2026-04-27	3003.64
8	8	49	111	2026-04-22	завершено	2026-05-15	6169.60
9	9	41	166	2026-04-22	завершено	2026-05-16	3834.30
10	10	30	14	2026-03-30	получено	2026-05-19	4603.04

+ Таблица order_item:

```
3 INSERT INTO publishing.order_item (  
4     id_order,  
5     isbn,  
6     quantity,  
7     unit_price,  
8     total_amount  
9 )  
10 SELECT  
11     o.id_order,  
12     b.isbn,  
13     q.quantity,  
14     q.unit_price,  
15     q.quantity * q.unit_price  
16 FROM publishing.orders o  
17  
18 JOIN LATERAL (  
19     SELECT *  
20     FROM publishing.book  
21     ORDER BY random()  
22     LIMIT (FLOOR(random()*8) + 1)::int  
23 ) b ON TRUE  
24  
25 CROSS JOIN LATERAL (  
26     SELECT  
27         (FLOOR(random()*10) + 1)::int AS quantity,  
28         ROUND((random()*900 + 100)::numeric, 2) AS unit_price  
29 ) q;  
30
```

	id_order [PK] integer	isbn [PK] character (13)	quantity integer	unit_price numeric	totalAmount numeric
1	1	ISBN-29	4	108.25	433.00
2	1	ISBN-354	4	108.25	433.00
3	1	ISBN-385	4	108.25	433.00
4	1	ISBN-157	4	108.25	433.00
5	2	ISBN-29	4	108.25	433.00
6	2	ISBN-354	4	108.25	433.00
7	2	ISBN-385	4	108.25	433.00
8	2	ISBN-157	4	108.25	433.00
9	3	ISBN-29	4	108.25	433.00
10	3	ISBN-354	4	108.25	433.00
11	3	ISBN-385	4	108.25	433.00
12	3	ISBN-157	4	108.25	433.00
13	4	ISBN-29	4	108.25	433.00
14	4	ISBN-354	4	108.25	433.00
15	4	ISBN-385	4	108.25	433.00
16	4	ISBN-157	4	108.25	433.00
17	5	ISBN-29	4	108.25	433.00
18	5	ISBN-354	4	108.25	433.00
19	5	ISBN-385	4	108.25	433.00
20	5	ISBN-157	4	108.25	433.00
21	6	ISBN-29	4	108.25	433.00
22	6	ISBN-354	4	108.25	433.00
23	6	ISBN-385	4	108.25	433.00
24	6	ISBN-157	4	108.25	433.00
25	7	ISBN-29	4	108.25	433.00
26	7	ISBN-354	4	108.25	433.00
27	7	ISBN-385	4	108.25	433.00
28	7	ISBN-157	4	108.25	433.00

+ Таблица circulation:

```
3  INSERT INTO publishing.circulation (  
4      isbn,  
5      print_type,  
6      circulation_quantity,  
7      circulation_date,  
8      notes  
9  )  
10 SELECT  
11     b.isbn,  
12  
13     CASE  
14         WHEN random() < 0.5 THEN 'оригинальный'  
15         ELSE 'дополнительный'  
16     END,  
17  
18     (floor(random()*4900) + 100)::int,  
19  
20     CURRENT_DATE - (floor(random()*1500))::int,  
21  
22     CASE  
23         WHEN random() < 0.3 THEN 'reprint'  
24         ELSE NULL  
25     END  
26  
27 FROM publishing.book b,  
28  
29 LATERAL generate_series(1, (floor(random()*3)+1)::int) g;  
30  
31 SELECT * FROM publishing.circulation LIMIT 20;
```

	id_circulation [PK] integer	isbn character (13)	print_type character varying (50)	circulation_quantity integer	circulation_date date	notes character varying (300)
1	1	ISBN-1	дополнительный	3964	2023-03-03	[null]
2	2	ISBN-2	оригинальный	3866	2024-08-11	reprint
3	3	ISBN-3	оригинальный	1189	2026-02-25	[null]
4	4	ISBN-4	дополнительный	2069	2024-03-08	[null]
5	5	ISBN-5	дополнительный	3238	2024-03-10	[null]
6	6	ISBN-6	оригинальный	2943	2022-08-18	[null]
7	7	ISBN-7	оригинальный	1570	2025-05-28	[null]
8	8	ISBN-8	оригинальный	950	2022-08-05	[null]
9	9	ISBN-9	дополнительный	3731	2022-10-01	[null]
10	10	ISBN-10	дополнительный	4184	2025-11-05	[null]
11	11	ISBN-11	дополнительный	2200	2026-04-15	[null]
12	12	ISBN-12	дополнительный	1416	2023-08-30	[null]
13	13	ISBN-13	дополнительный	171	2024-02-08	reprint
14	14	ISBN-14	дополнительный	1010	2023-09-28	[null]
15	15	ISBN-15	дополнительный	2663	2023-04-13	[null]
16	16	ISBN-16	дополнительный	1943	2024-02-24	reprint
17	17	ISBN-17	оригинальный	114	2022-11-15	[null]
18	18	ISBN-18	оригинальный	4553	2024-12-01	[null]
19	19	ISBN-19	оригинальный	4569	2024-02-29	[null]
20	20	ISBN-20	оригинальный	1074	2022-06-13	[null]

+ Таблица technical_assignment:

Query	Query History
2	
3	INSERT INTO publishing.technical_assignment (
4	isbn,
5	id_author_ta,
6	has_illustration,
7	binding_type,
8	created_date,
9	paper_type,
10	notes
11	)
12	SELECT
13	b.isbn,
14	
15	-- random author
16	a.id_author,
17	
18	-- illustration
19	(random() < 0.5) AS has_illustration,
20	
21	-- binding type
22	CASE
23	WHEN random() < 0.5 THEN 'твердый'
24	ELSE 'мягкий'
25	END AS binding_type,
26	
27	-- created date <= today
28	CURRENT_DATE - (floor(random()*900))::int AS created_date,
29	
30	-- paper type (CHECK-safe)
31	CASE
32	WHEN random() < 0.5 THEN 'офсетная'
33	ELSE 'мелованная'
34	END AS paper_type,
35	
36	-- notes
37	CASE
38	WHEN random() < 0.3 THEN 'layout approved'
39	WHEN random() < 0.6 THEN 'needs revision'
40	ELSE NULL
41	END AS notes
42	
	FROM publishing.book b
	JOIN LATERAL generate_series(1, (floor(random()*2)+1)::int) gs ON TRUE
	JOIN LATERAL (
	SELECT id_author
	FROM publishing.author
	ORDER BY random()
	LIMIT 1
	) a ON TRUE;
	SELECT * FROM publishing.technical_assignment LIMIT 20;

Data Output Messages Notifications									
Showing rows: 1									
	id_ta [PK] integer	isbn character (13)	id_author_ta integer	has_illustration boolean	binding_type character varying (100)	created_date date	paper_type character varying (100)	notes character varying (300)	
1	1	ISBN-1	6	true	мягкий	2026-02-07	мелованная	needs revision	
2	2	ISBN-2	6	true	твердый	2025-09-04	мелованная	[null]	
3	3	ISBN-3	6	true	мягкий	2024-12-28	офсетная	[null]	
4	4	ISBN-4	6	false	твердый	2025-12-19	офсетная	layout approved	
5	5	ISBN-5	6	true	мягкий	2024-11-12	мелованная	layout approved	
6	6	ISBN-6	6	true	мягкий	2026-03-02	мелованная	[null]	
7	7	ISBN-7	6	false	твердый	2025-01-25	мелованная	needs revision	
8	8	ISBN-8	6	true	твердый	2024-03-03	мелованная	needs revision	
9	9	ISBN-9	6	false	твердый	2025-07-31	офсетная	[null]	
10	10	ISBN-10	6	true	мягкий	2025-10-31	мелованная	needs revision	
11	11	ISBN-11	6	false	мягкий	2023-12-11	офсетная	needs revision	
12	12	ISBN-12	6	true	твердый	2026-04-08	офсетная	[null]	
13	13	ISBN-13	6	true	твердый	2025-12-15	мелованная	[null]	
14	14	ISBN-14	6	true	мягкий	2024-01-23	офсетная	[null]	
15	15	ISBN-15	6	false	мягкий	2025-11-14	мелованная	layout approved	
16	16	ISBN-16	6	false	твердый	2026-02-21	мелованная	layout approved	
17	17	ISBN-17	6	false	мягкий	2026-01-10	мелованная	layout approved	
18	18	ISBN-18	6	true	мягкий	2025-06-17	офсетная	layout approved	
19	19	ISBN-19	6	false	мягкий	2025-10-22	мелованная	[null]	
20	20	ISBN-20	6	true	мягкий	2025-07-25	офсетная	layout approved	

+ Таблица editing_ta:

Query	Query History
1	INSERT INTO publishing.editing_ta (
2	id_ta,
3	id_editor,
4	participation_start,
5	order_editor,
6	main_editor,
7	comment
8	)
9	SELECT ta.id_ta, e.id_employee,
10	
11	CURRENT_DATE - (floor(random()*800) + gs.editor_order * 7)::int, gs.editor_order,
12	
13	CASE WHEN gs.editor_order = 1 THEN TRUE ELSE FALSE END,
14	
15	CASE
16	WHEN gs.editor_order = 1 THEN 'lead editor'
17	WHEN gs.editor_order = 2 THEN 'review editor'
18	ELSE 'assistant editor'
19	END
20	
21	FROM publishing.technical_assignment ta
22	
23	JOIN LATERAL generate_series(1, (floor(random()*3)+1)::int) gs(editor_order) ON TRUE
24	
25	JOIN LATERAL (
26	SELECT id_employee
27	FROM publishing.employee
28	ORDER BY md5(random()::text ta.id_ta::text gs.editor_order::text)
29	LIMIT 1
30	) e ON TRUE;
31	

Data Output Messages Notifications							
	id_editing [PK] integer	id_ta integer	id_editor integer	participation_start date	order_editor character varying (100)	main_editor character varying (100)	comment character varying (300)
1	1	1	14	2025-05-17	1	true	lead editor
2	2	1	8	2025-12-02	2	false	review editor
3	3	2	28	2025-02-10	1	true	lead editor
4	4	2	14	2026-01-16	2	false	review editor
5	5	3	9	2025-05-24	1	true	lead editor
6	6	3	26	2024-10-29	2	false	review editor
7	7	4	18	2024-03-16	1	true	lead editor
8	8	4	10	2025-04-18	2	false	review editor
9	9	5	7	2024-06-17	1	true	lead editor
10	10	5	19	2025-12-13	2	false	review editor
11	11	6	6	2025-01-31	1	true	lead editor
12	12	6	13	2024-12-10	2	false	review editor
13	13	7	23	2024-12-26	1	true	lead editor
14	14	7	28	2025-05-01	2	false	review editor
15	15	8	17	2024-04-04	1	true	lead editor
16	16	8	35	2024-07-05	2	false	review editor
17	17	9	47	2026-02-22	1	true	lead editor
18	18	9	19	2024-09-26	2	false	review editor
19	19	10	31	2026-02-02	1	true	lead editor

+ Таблица contract:

Query Query History	
3	INSERT INTO publishing.contract (
4	id_order,
5	id_manager,
6	id_customer,
7	contract_status,
8	contract_date,
9	copies
10	)
11	SELECT
12	o.id_order,
13	m.id_employee,
14	o.id_customer,
15	
16	-- status
17	CASE
18	WHEN random() < 0.5 THEN 'активный'
19	WHEN random() < 0.8 THEN 'закрытый'
20	ELSE 'ожидающий'
21	END AS contract_status,
22	
23	o.order_date - INTERVAL '1 day' * floor(random()*10),
24	
25	-- fixed constraint
26	2 AS copies
27	
28	FROM publishing.orders o
29	
30	JOIN LATERAL (
31	SELECT id_employee
32	FROM publishing.employee
33	ORDER BY md5(o.id_order::text random()::text)
34	LIMIT 1
35	) m ON TRUE;
36	
37	SELECT * FROM publishing.contract LIMIT 30;

	id_contract [PK] integer	id_order integer	id_manager integer	id_customer integer	contract_status character varying (50)	contract_date date	copies integer
1	1	1	16	189	активный	2026-04-20	2
2	2	2	19	24	активный	2026-04-16	2
3	3	3	10	153	закрытый	2026-04-15	2
4	4	4	32	55	ожидающий	2026-03-31	2
5	5	5	47	160	активный	2026-03-31	2
6	6	6	27	15	ожидающий	2026-04-14	2
7	7	7	30	196	ожидающий	2026-03-19	2
8	8	8	6	111	активный	2026-04-17	2
9	9	9	31	166	активный	2026-04-13	2
10	10	10	19	14	активный	2026-03-21	2
11	11	11	30	151	закрытый	2026-03-30	2
12	12	12	12	150	закрытый	2026-03-25	2
13	13	13	19	58	активный	2026-04-01	2
14	14	14	18	169	закрытый	2026-04-15	2
15	15	15	47	59	закрытый	2026-03-23	2
16	16	16	24	51	закрытый	2026-04-13	2
17	17	17	38	102	активный	2026-04-11	2
18	18	18	18	134	активный	2026-04-06	2
19	19	19	50	111	закрытый	2026-04-18	2
20	20	20	6	191	активный	2026-03-23	2

+ Таблица act:

```

3  INSERT INTO publishing.act (
4      id_contract,
5      sign_date,
6      copies,
7      notes
8  )
9  SELECT
10     c.id_contract,
11
12     LEAST(
13         c.contract_date + (floor(random()*7))::int,
14         CURRENT_DATE
15     ) AS sign_date,
16
17     2,
18
19     CASE
20         WHEN c.contract_status = 'активный' THEN 'active contract signed'
21         WHEN c.contract_status = 'ожидающий' THEN 'pending approval'
22         ELSE 'closed contract'
23     END
24
25 FROM publishing.contract c;
26
27 SELECT * FROM publishing.act LIMIT 20;
28

```

	id_act [PK] integer	id_contract integer	sign_date date	copies integer	notes character varying (300)
1	1	1	2026-04-23	2	active contract signed
2	2	2	2026-04-17	2	active contract signed
3	3	3	2026-04-17	2	closed contract
4	4	4	2026-04-05	2	pending approval
5	5	5	2026-04-02	2	active contract signed
6	6	6	2026-04-19	2	pending approval
7	7	7	2026-03-21	2	pending approval
8	8	8	2026-04-21	2	active contract signed
9	9	9	2026-04-15	2	active contract signed
10	10	10	2026-03-22	2	active contract signed
11	11	11	2026-04-04	2	closed contract
12	12	12	2026-03-29	2	closed contract
13	13	13	2026-04-04	2	active contract signed
14	14	14	2026-04-20	2	closed contract
15	15	15	2026-03-23	2	closed contract
16	16	16	2026-04-13	2	closed contract
17	17	17	2026-04-12	2	active contract signed
18	18	18	2026-04-09	2	active contract signed
19	19	19	2026-04-23	2	closed contract
20	20	20	2026-03-23	2	active contract signed

+ Таблица invoice:

```
3  INSERT INTO publishing.invoice (  
4      id_contract,  
5      invoice_type,  
6      sign_date,  
7      amount,  
8      payment_status  
9  )  
10 SELECT  
11     a.id_contract,  
12  
13     CASE  
14         WHEN random() < 0.5 THEN 'предоплата'  
15         ELSE 'частичная оплата'  
16     END AS invoice_type,  
17  
18     -- sign_date  
19     a.sign_date AS sign_date,  
20  
21     -- amount  
22     ROUND(  
23         (  
24             (floor(random()*5) + 1) * 1000  
25         )::numeric,  
26         2  
27     ) AS amount,  
28  
29     -- status  
30     CASE  
31         WHEN random() < 0.6 THEN TRUE  
32         ELSE FALSE  
33     END AS payment_status  
34  
35 FROM publishing.act a;  
36  
37 SELECT * FROM publishing.invoice LIMIT 20;
```

	id_invoice [PK] integer	id_contract integer	invoice_type character varying (50)	sign_date date	amount numeric	payment_status boolean
1	1	1	предоплата	2026-04-23	3000.00	false
2	2	2	предоплата	2026-04-17	1000.00	true
3	3	3	частичная оплата	2026-04-17	3000.00	true
4	4	4	предоплата	2026-04-05	3000.00	true
5	5	5	частичная оплата	2026-04-02	5000.00	false
6	6	6	частичная оплата	2026-04-19	2000.00	false
7	7	7	предоплата	2026-03-21	5000.00	false
8	8	8	частичная оплата	2026-04-21	5000.00	false
9	9	9	предоплата	2026-04-15	2000.00	false
10	10	10	частичная оплата	2026-03-22	4000.00	false
11	11	11	предоплата	2026-04-04	5000.00	false
12	12	12	предоплата	2026-03-29	3000.00	false
13	13	13	частичная оплата	2026-04-04	4000.00	false
14	14	14	предоплата	2026-04-20	5000.00	true
15	15	15	частичная оплата	2026-03-23	2000.00	true
16	16	16	предоплата	2026-04-13	4000.00	true
17	17	17	частичная оплата	2026-04-12	5000.00	true
18	18	18	частичная оплата	2026-04-09	5000.00	true
19	19	19	предоплата	2026-04-23	2000.00	true
20	20	20	предоплата	2026-03-23	5000.00	false

– Потом были разработаны SQL-запросы для выборки данных. Запросы включали:

- фильтрацию данных (WHERE);
- сортировку (ORDER BY);
- соединение таблиц (JOIN);
- агрегатные функции (COUNT, SUM);
- группировку (GROUP BY).

+ Пример 1:

Query

Query History

1

2

3

4

5

6

7

8

SELECT

o.id_order,

c.company_name,

o.order_date,

o.status,

o.total_amount

FROM publishing.orders o

JOIN publishing.customer c ON o.id_customer = c.id_customer;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	id_order integer	company_name character varying (200)	order_date date	status character varying (50)	total_amount numeric
1	1	Company 189	2026-04-22	завершено	9718.77
2	2	Company 24	2026-04-20	отменено	6334.91
3	3	Company 153	2026-04-21	доставлено	1455.78
4	4	Company 55	2026-03-31	обработано	1282.17
5	5	Company 160	2026-04-05	отменено	3292.20
6	6	Company 15	2026-04-23	доставлено	5826.91
7	7	Company 196	2026-03-26	получено	3003.64
8	8	Company 111	2026-04-22	завершено	6169.60
9	9	Company 166	2026-04-22	завершено	3834.30
10	10	Company 14	2026-03-30	получено	4603.04
11	11	Company 151	2026-04-04	отменено	2908.39
12	12	Company 150	2026-03-27	отменено	6962.20
13	13	Company 58	2026-04-08	доставлено	337.59
14	14	Company 169	2026-04-19	отменено	1064.28
15	15	Company 59	2026-03-31	завершено	9915.25
16	16	Company 51	2026-04-14	отменено	5657.13
17	17	Company 102	2026-04-20	доставлено	3123.48
18	18	Company 134	2026-04-07	доставлено	769.64
19	19	Company 111	2026-04-19	отменено	6016.84
20	20	Company 191	2026-03-26	отменено	1526.60

+ Пример 2:

Query

Query History

1

SELECT

2

o.id_order,

3

b.title,

4

oi.quantity,

5

oi.unit_price,

6

oi.total_amount

7

FROM publishing.order_item oi

8

JOIN publishing.orders o ON oi.id_order = o.id_order

9

JOIN publishing.book b ON oi.isbn = b.isbn;

Data Output

Messages

Notifications

+ Пример 3:

Query

Query History

1

2

3

4

5

6

SELECT

c.company_name,

SUM(o.total_amount) AS total_spent

FROM publishing.orders o

JOIN publishing.customer c ON o.id_customer = c.id_customer

GROUP BY c.company_name;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	company_name character varying (200)	total_spent numeric
1	Company 112	31055.67
2	Company 133	1694.71
3	Company 95	27315.98
4	Company 184	9421.37
5	Company 136	2607.47
6	Company 53	6441.91
7	Company 140	13485.96
8	Company 137	679.89
9	Company 40	9775.49
10	Company 70	5132.21
11	Company 199	7628.61
12	Company 21	7695.50
13	Company 151	20685.01
14	Company 18	8155.83
15	Company 90	9812.64
16	Company 86	3255.52
17	Company 157	4649.63
18	Company 24	14448.73
19	Company 179	6402.19
20	Company 61	19024.73
21	Company 193	3154.56
22	Company 160	9693.25

– Создание представления VIEW

Query Query History

```
1 CREATE VIEW publishing.v_order_detail AS
2 SELECT
3     o.id_order,
4     c.company_name,
5     o.order_date,
6     o.status,
7     o.total_amount
8 FROM publishing.orders o
9 JOIN publishing.customer c ON o.id_customer = c.id_customer;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 154 msec.

Query Query History

```
1 SELECT *
2 FROM publishing.v_order_detail;
```

Data Output Messages Notifications

≡+

▼

▼

SQL

	id_order integer	company_name character varying (200)	order_date date	status character varying (50)	total_amount numeric
1	1	Company 189	2026-04-22	завершено	9718.77
2	2	Company 24	2026-04-20	отменено	6334.91
3	3	Company 153	2026-04-21	доставлено	1455.78
4	4	Company 55	2026-03-31	обработано	1282.17
5	5	Company 160	2026-04-05	отменено	3292.20
6	6	Company 15	2026-04-23	доставлено	5826.91
7	7	Company 196	2026-03-26	получено	3003.64
8	8	Company 111	2026-04-22	завершено	6169.60
9	9	Company 166	2026-04-22	завершено	3834.30
10	10	Company 14	2026-03-30	получено	4603.04
11	11	Company 151	2026-04-04	отменено	2908.39
12	12	Company 150	2026-03-27	отменено	6962.20
13	13	Company 58	2026-04-08	доставлено	337.59
14	14	Company 169	2026-04-19	отменено	1064.28
15	15	Company 59	2026-03-31	завершено	9915.25
16	16	Company 51	2026-04-14	отменено	5657.13
17	17	Company 102	2026-04-20	доставлено	3123.48
18	18	Company 134	2026-04-07	доставлено	769.64
19	19	Company 111	2026-04-19	отменено	6016.84
20	20	Company 191	2026-03-26	отменено	1526.60

4.2 Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.

– Запроса 1: INSERT

Query

Query History

1

```
INSERT INTO publishing.customer ( id_customer, contact_legal_entity, company_name, id_tax, phone, address, email )
```

2

```
SELECT 999, 'Test Person', 'Test Company', '999999999', '000000', 'Test Address', 'test@mail.com'
```

3

```
WHERE NOT EXISTS (SELECT 1 FROM publishing.customer WHERE id_customer = 999 );
```

Data Output

Messages

Notifications

INSERT 0 1

Query returned successfully in 72 msec.

Мы проверяем:

Query

Query History

1

```
SELECT COUNT(*) FROM publishing.customer WHERE id_customer = 999
```

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

	count bigint
1	1

Query

Query History

1

```
SELECT * FROM publishing.customer WHERE id_customer = 999
```

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

	id_customer [PK] integer	contact_legal_entity character varying (200)	company_name character varying (200)	id_tax character varying (10)	phone character varying (20)	address character varying (300)	email character varying (200)
1	999	Test Person	Test Company	9999999999	000000	Test Address	test@mail.com

– Запроса 2: UPDATE

+ Перед выполнением запроса посмотрим затронутые данные:

Query

Query History

```
1 SELECT *
2 FROM publishing.book
3 WHERE id_category IN (
4     SELECT id_category
5     FROM publishing.category
6     WHERE category_name = 'Programming'
7 );
```

Data Output

Messages

Notifications

Showing rows: 1 to 63

Page No: 1

	isbn [PK] character varying	id_category integer	title character varying (200)	type character varying (50)	edition_number integer	num_pages integer	stock_quantity integer	retail_price numeric	has_illustration boolean	year_first_publish integer
1	ISBN-2	1	Book Title 2	учебное пособие	4	511	34	387.992	false	2021
2	ISBN-11	1	Book Title 11	учебник	4	318	108	345.774	false	2009
3	ISBN-12	1	Book Title 12	учебное пособие	4	448	84	155.925	true	2009
4	ISBN-21	1	Book Title 21	учебник	3	235	71	194.612	true	2009
5	ISBN-30	1	Book Title 30	учебное пособие	3	276	110	746.801	true	2019
6	ISBN-35	1	Book Title 35	учебное пособие	1	331	47	770.627	true	2019
7	ISBN-50	1	Book Title 50	учебное пособие	3	398	64	978.736	false	2010
8	ISBN-58	1	Book Title 58	учебное пособие	4	249	22	274.901	true	2023
9	ISBN-72	1	Book Title 72	учебное пособие	2	123	23	561.462	true	2015
10	ISBN-73	1	Book Title 73	учебник	3	538	21	1000.054	true	2017
11	ISBN-90	1	Book Title 90	учебное пособие	5	596	86	921.415	true	2012
12	ISBN-91	1	Book Title 91	учебное пособие	4	315	92	1055.714	true	2009
13	ISBN-92	1	Book Title 92	учебник	4	273	38	193.996	false	2022
14	ISBN-93	1	Book Title 93	учебник	2	588	50	1027.114	true	2014
15	ISBN-107	1	Book Title 107	учебник	1	597	30	1091.266	true	2010
16	ISBN-119	1	Book Title 119	учебник	5	303	99	851.851	false	2022
17	ISBN-120	1	Book Title 120	учебное пособие	3	497	23	328.020	false	2006
18	ISBN-127	1	Book Title 127	учебное пособие	3	306	12	139.557	false	2024
19	ISBN-141	1	Book Title 141	учебное пособие	4	257	72	279.565	false	2012
20	ISBN-158	1	Book Title 158	учебное пособие	1	387	13	817.300	true	2010

+ Проверим цены книга:

Query Query History	
1	SELECT isbn, retail_price
2	FROM publishing.book
3	WHERE id_category IN (
4	SELECT id_category
5	FROM publishing.category
6	WHERE category_name = 'Programming'
7	);

Data Output Messages Notifications	
≡ 📄 ▼ 📋 ▼ 🗑 📦 ⬇ 📈 SQL	
	isbn [PK] character varying
	retail_price numeric
1	ISBN-2 581.9880
2	ISBN-11 518.6610
3	ISBN-12 233.8875
4	ISBN-21 291.9180
5	ISBN-30 1120.2015
6	ISBN-35 1155.9405
7	ISBN-50 1468.1040
8	ISBN-58 412.3515
9	ISBN-72 842.1930
10	ISBN-73 1500.0810
11	ISBN-90 1382.1225
12	ISBN-91 1583.5710
13	ISBN-92 290.9940
14	ISBN-93 1540.6710
15	ISBN-107 1636.8990
16	ISBN-119 1277.7765
17	ISBN-120 492.0300
18	ISBN-127 209.3355
19	ISBN-141 419.3475
20	ISBN-158 1225.9500

+ Используем запрос UPDATE для изменения цен на книги в категории Программирование.

Query Query History	
1	UPDATE publishing.book
2	SET retail_price = retail_price * 1.5
3	WHERE id_category IN (SELECT id_category FROM publishing.category WHERE category_name = 'Programming');

Data Output Messages Notifications	
UPDATE 63	
Query returned successfully in 80 msec.	

+ Проверим цены книга после изменения :

Query Query History

```
1 SELECT isbn, retail_price
2 FROM publishing.book
3 WHERE id_category IN (
4     SELECT id_category
5     FROM publishing.category
6     WHERE category_name = 'Programming'
7 );
```

[Data Output](#) [Messages](#) [Notifications](#)

	isbn [PK] character varying	retail_price numeric
1	ISBN-2	581.9880
2	ISBN-11	518.6610
3	ISBN-12	233.8875
4	ISBN-21	291.9180
5	ISBN-30	1120.2015
6	ISBN-35	1155.9405
7	ISBN-50	1468.1040
8	ISBN-58	412.3515
9	ISBN-72	842.1930
10	ISBN-73	1500.0810
11	ISBN-90	1382.1225
12	ISBN-91	1583.5710
13	ISBN-92	290.9940
14	ISBN-93	1540.6710
15	ISBN-107	1636.8990
16	ISBN-119	1277.7765
17	ISBN-120	492.0300
18	ISBN-127	209.3355
19	ISBN-141	419.3475
20	ISBN-158	1225.9500

– Запроса 3: DELETE

- + Перед выполнением запроса проверить данные с contract_status “ожидающий” в таблице invoice.

Query		Query History	
1	SELECT *		
2	FROM publishing.invoice		
3	WHERE id_contract IN (SELECT id_contract FROM publishing.contract WHERE contract_status = 'ожидающий')		
4	ORDER BY id_invoice		
5	);		
6			

Data Output		Messages		Notifications	
id_invoice	id_contract	invoice_type	sign_date	amount	payment_status
[PK] integer	integer	character varying (50)	date	numeric	boolean
1	200	частичная оплата	2026-04-02	4000.00	true
2	201	предоплата	2026-04-08	4000.00	false
3	205	предоплата	2026-04-12	3000.00	false
4	226	предоплата	2026-04-07	1000.00	true
5	232	частичная оплата	2026-03-19	4000.00	true
6	239	предоплата	2026-04-03	3000.00	true
7	253	частичная оплата	2026-04-05	2000.00	true
8	275	частичная оплата	2026-04-12	5000.00	true
9	294	предоплата	2026-04-18	5000.00	false
10	296	предоплата	2026-04-20	3000.00	true

- + Используем запрос DELETE для удаления 2 invoice с contract_status “ожидающий”.

Query	Query History
1 2 3 4 5 6	<pre>DELETE FROM publishing.invoice WHERE id_invoice IN (SELECT id_invoice FROM publishing.invoice i WHERE id_contract IN (SELECT id_contract FROM publishing.contract WHERE contract_status = 'ожидающий') ORDER BY id_invoice LIMIT 2);</pre>

Data Output	Messages	Notifications
DELETE 2		
Query returned successfully in 87 msec.		

- + Проверяем, что 2 первые invoice с contract_status “ожидающий” были удалены:

Query

Query History

1

SELECT

*

FROM

publishing.invoice

2

WHERE

id_contract

IN

(

3

SELECT

id_contract

FROM

publishing.contract

WHERE

contract_status

=

'ожидающий'

4

ORDER

BY

id_invoice

)

;

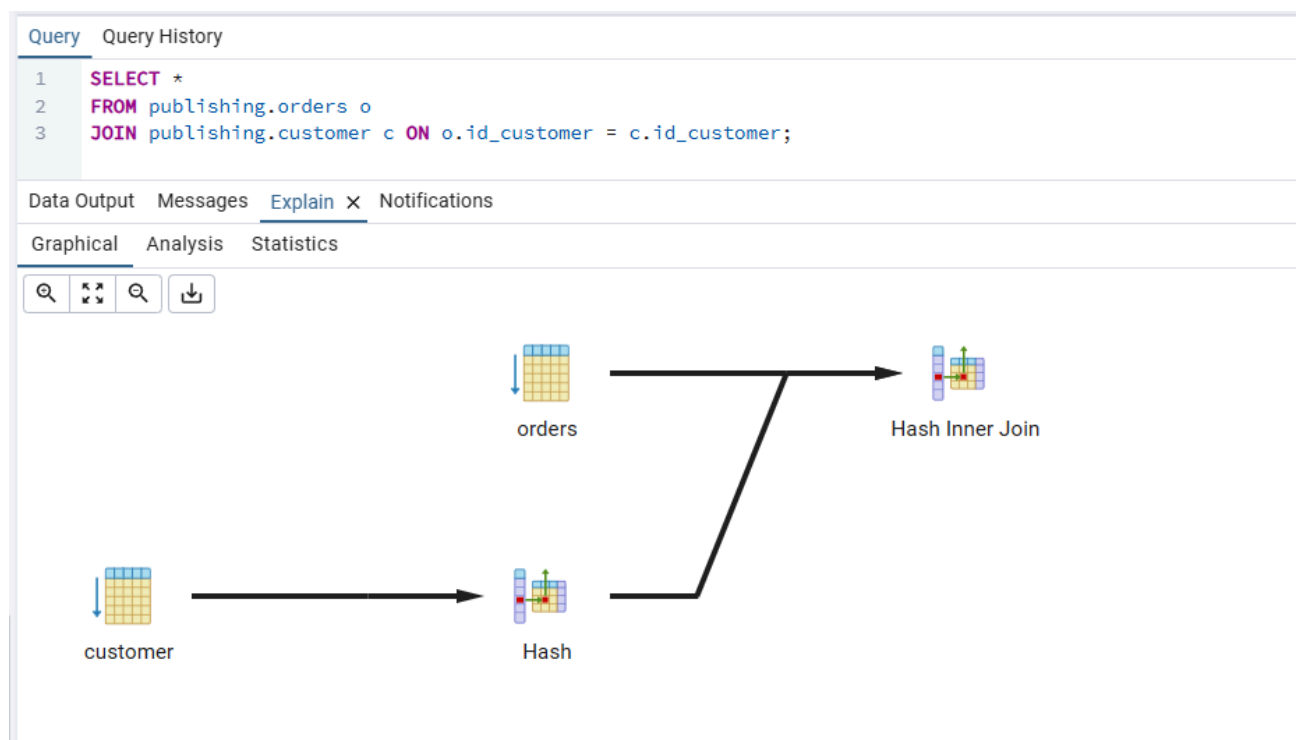
5

Data Output

Messages

Notifications

4.3 Изучить графическое представление запросов и просмотреть историю запросов.



→ Этот графический план выполнения показывает, как база данных оптимизирует операцию JOIN между таблицами *orders* и *customer*.

– Схема демонстрирует алгоритм Hash Join:

- + База данных сначала сканирует таблицу *customer*.
- + Затем она создает хеш-таблицу в оперативной памяти на основе столбца *id_customer*

- + После того как хеш-таблица готова, база данных сканирует таблицу *orders*.
- + Для каждой строки из *orders* она вычисляет хеш-значение для *id_customer* и ищет совпадение в уже созданной хеш-таблице *customer*.
- + Сопоставленные строки объединяются и возвращаются как результат запроса.

4.4 Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

+ Запрос 1: простой индекс

Copy Copy to Query Editor

```
CREATE INDEX idx_orders_amount ON publishing.orders(total_amount);
```

Messages

Query returned successfully in 69 msec.

До использования индексов:

Query
Query History

```

1  EXPLAIN ANALYZE
2  SELECT *
3  FROM publishing.orders
4  ORDER BY total_amount;

```

Data Output
Messages
Notifications

Showing rows: 1 to 10
Page No: 1 of 1

	QUERY PLAN
1	Sort (cost=18.34..19.09 rows=300 width=45) (actual time=1.282..1.293 rows=300.00 loops=1)
2	Sort Key: total_amount
3	Sort Method: quicksort Memory: 44kB
4	Buffers: shared hit=3 read=3
5	-> Seq Scan on orders (cost=0.00..6.00 rows=300 width=45) (actual time=0.794..0.922 rows=300.00 loop...
6	Buffers: shared read=3
7	Planning:
8	Buffers: shared hit=117 read=8
9	Planning Time: 5.458 ms
10	Execution Time: 1.329 ms

После использования индексов:

The screenshot shows the PostgreSQL query editor interface. The query entered is:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM publishing.orders
4 ORDER BY total_amount;
5
```

The results pane shows the query plan:

QUERY PLAN	
	text
1	Index Scan using idx_orders_amount on orders (cost=0.15..24.65 rows=300 width=45) (actual time=0.072..0.161 rows=300.00 loo...
2	Index Searches: 1
3	Buffers: shared hit=181 read=1
4	Planning Time: 0.116 ms
5	Execution Time: 0.193 ms

→ До создания индекса запрос выполнялся с использованием последовательного сканирования (Seq Scan) и дополнительной операции сортировки, что увеличивало время выполнения. После создания индекса и его принудительного использования PostgreSQL перешёл к Index Scan, что позволило избежать сортировки и значительно сократить время выполнения запроса (1.329 ms → 0.193 ms).

+ Запрос 2: составной индекс

До использования индексов:

The screenshot shows the PostgreSQL query editor interface. The query entered is:

```
1 CREATE INDEX idx_orders_status_amount
2 ON publishing.orders(status, total_amount);
```

The results pane shows the message:

```
CREATE INDEX

Query returned successfully in 79 msec.
```


Query

Query History

1

EXPLAIN ANALYZE

2

SELECT * FROM publishing.orders

3

WHERE status = 'завершено' AND total_amount BETWEEN 1000 AND 5000;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 8Page No: 1 of 1

QUERY PLAN

text

1

Seq Scan on orders (cost=0.00..8.25 rows=21 width=45) (actual time=0.019..0.082 rows=28.00 loops=1)

2

Filter: ((total_amount >= '1000'::numeric) AND (total_amount <= '5000'::numeric) AND ((status)::text = 'завершено'::...)

3

Rows Removed by Filter: 272

4

Buffers: shared hit=3 dirtied=1

5

Planning:

6

Buffers: shared hit=5 dirtied=1

7

Planning Time: 0.585 ms

8

Execution Time: 0.097 ms

После использования индексов:

Query Query History	
1	EXPLAIN ANALYZE
2	SELECT * FROM publishing.orders
3	WHERE status = 'завершено' AND total_amount BETWEEN 1000 AND 5000;
Data Output Messages Notifications	
≡ 📄 ▼ 🗑️ 📁 ⬇️ 📈 SQL Showing rows: 1 to 12 Page No: 1	
	QUERY PLAN text 🔒
1	Bitmap Heap Scan on orders (cost=4.54..7.91 rows=21 width=45) (actual time=0.099..0.106 rows=28.00 loops=1)
2	Recheck Cond: (((status)::text = 'завершено'::text) AND (total_amount >= '1000'::numeric) AND (total_amount <= '5000'::num...
3	Heap Blocks: exact=3
4	Buffers: shared hit=3 read=2
5	-> Bitmap Index Scan on idx_orders_status_amount (cost=0.00..4.54 rows=21 width=0) (actual time=0.054..0.054 rows=28....
6	Index Cond: (((status)::text = 'завершено'::text) AND (total_amount >= '1000'::numeric) AND (total_amount <= '5000'::num...
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=18 read=1
11	Planning Time: 0.992 ms
12	Execution Time: 0.125 ms

→ После создания индекса PostgreSQL перешёл от последовательного сканирования (Seq Scan) к Bitmap Index Scan. Однако время выполнения запроса увеличилось (0.097 ms → 0.125 ms), так как для небольшой таблицы дополнительные операции работы с индексом создают избыточные затраты. Это показывает, что использование индексов не всегда приводит к ускорению и зависит от объёма данных.

5. Выводы

В ходе выполнения лабораторной работы были изучены основные возможности работы с базой данных PostgreSQL. Были реализованы выборочные SQL-запросы различной сложности, позволяющие извлекать и анализировать данные в соответствии с индивидуальным заданием.

Также были созданы представления (VIEW), обеспечивающие удобный доступ к объединённым данным и упрощающие повторное использование запросов.

Особое внимание было уделено запросам на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов. Это позволило на практике освоить методы изменения данных с учётом связей между таблицами и условий выборки.

Кроме того, были изучены принципы оптимизации запросов с помощью индексов. Проведено сравнение планов выполнения запросов и времени их выполнения до и после создания индексов с использованием команды EXPLAIN ANALYZE. Было установлено, что эффективность индексов зависит от структуры запроса и объёма данных: в некоторых случаях использование

индекса ускоряет выполнение запроса, а в других – может приводить к дополнительным затратам.

Таким образом, в результате выполнения лабораторной работы были получены практические навыки работы с SQL-запросами, представлениями, подзапросами и индексами, а также понимание их влияния на производительность системы управления базами данных.